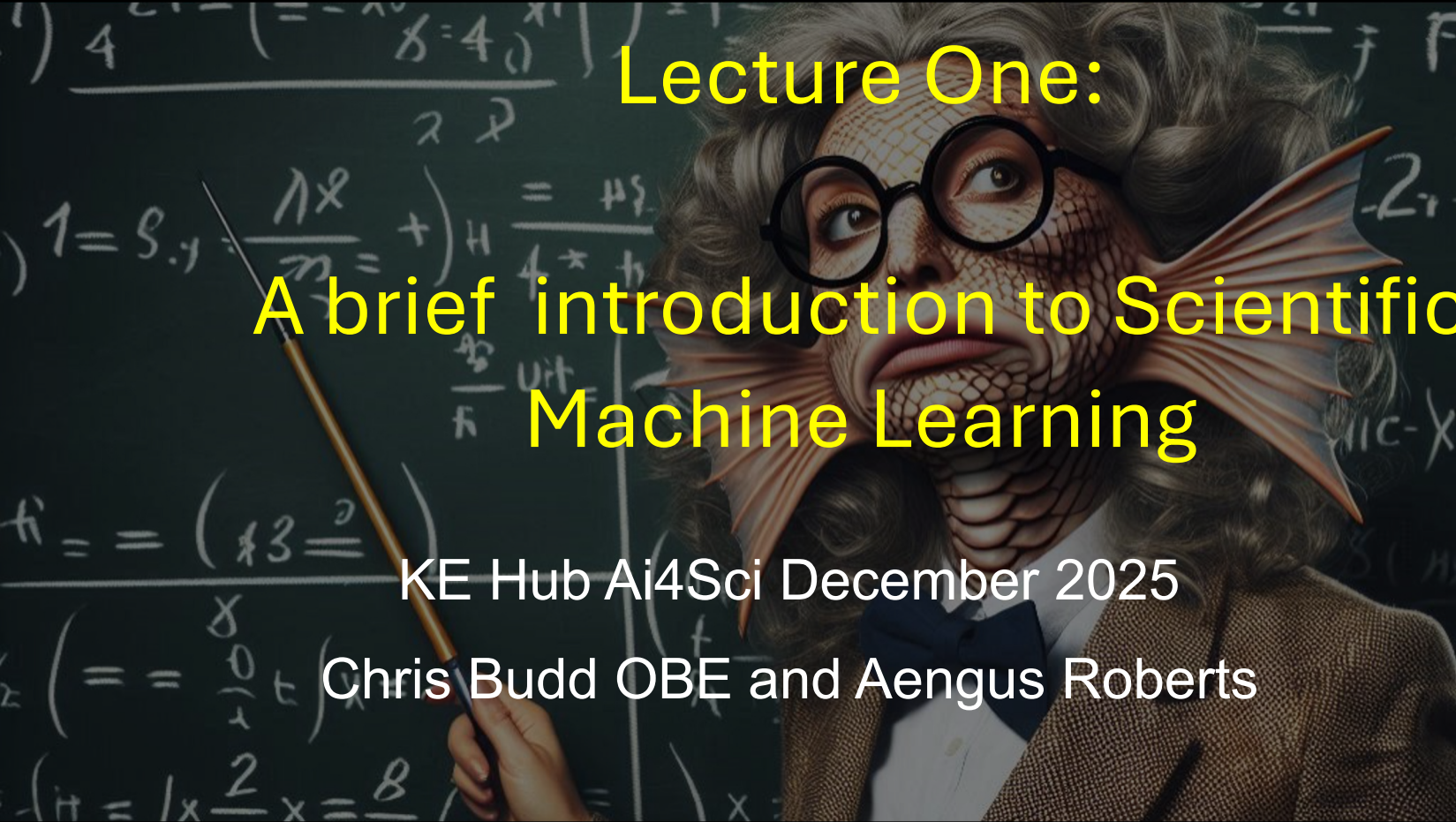
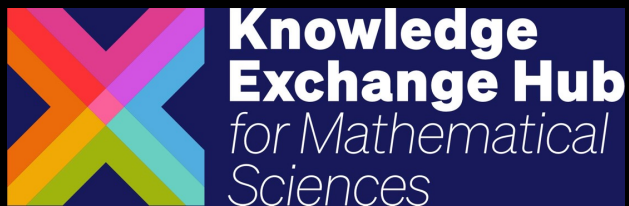




Lecture One: A brief introduction to Scientific Machine Learning

KE Hub Ai4Sci December 2025

Chris Budd OBE and Aengus Roberts



www.bbc.co.uk/news/science-environment-67383755

NEWS

Home | InDepth | Israel-Gaza war | War in Ukraine | Climate | UK | World | Business | Politics | Culture

Science & Environment

AI could predict hurricane landfall sooner - report

14 November 2023 · 45 Comments

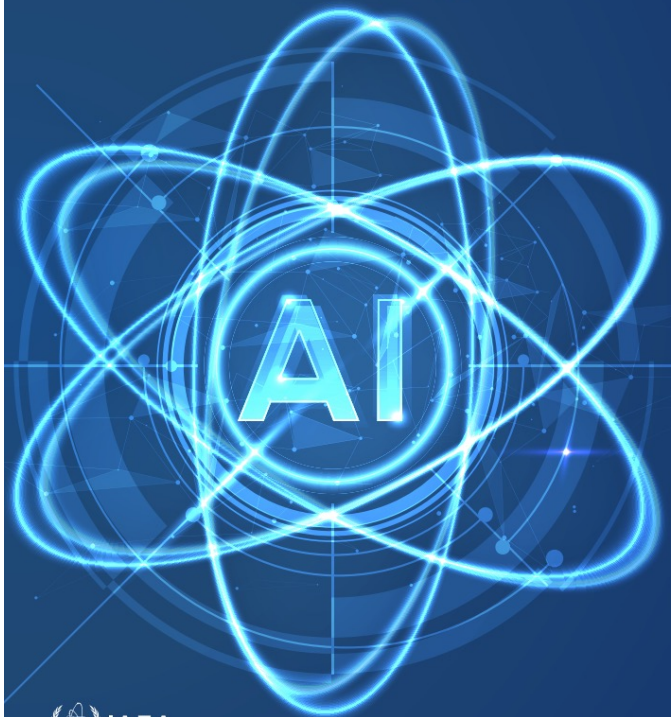


AI In the News

Makes 'better prediction of Hurricane Lee' than any other method

How good is the hype?

**Artificial Intelligence for
Accelerating Nuclear Applications,
Science and Technology**



Deep Learning



If you were ill, would you trust a diagnosis only made by a machine without involving a doctor?

Would you trust a weather forecast made by a machine without using the laws of physics?



Would you be happy to fly on aeroplane designed just from looking at previous aeroplanes?

All of the above are possible now, or in the near future, thanks to machine learning. But should we trust them?

Poster by
Helena
Lake

Structure of the mini-course

1. Introduction

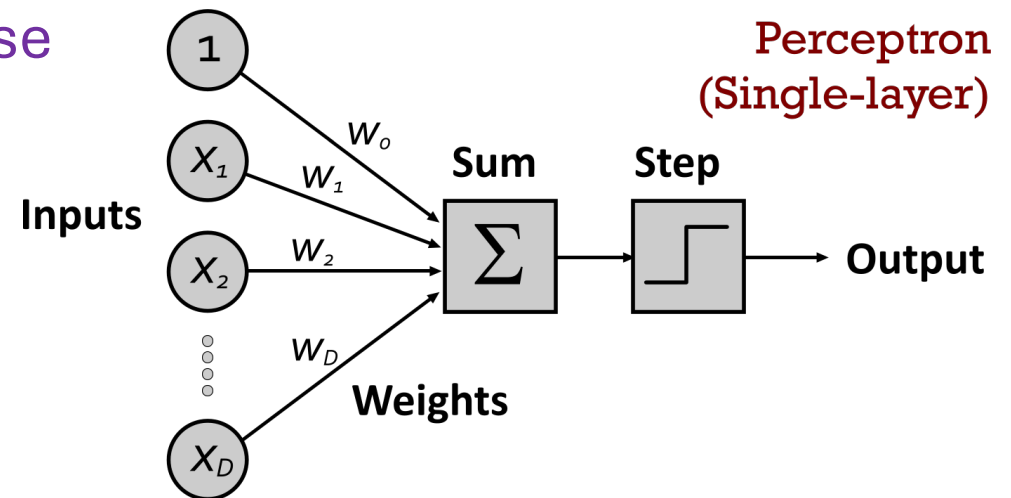
2. Architectures

3. ML for differential equations

4. Neural operators and the weather

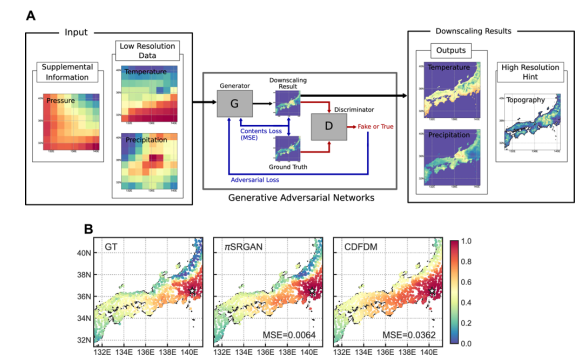
5. Neural ODES

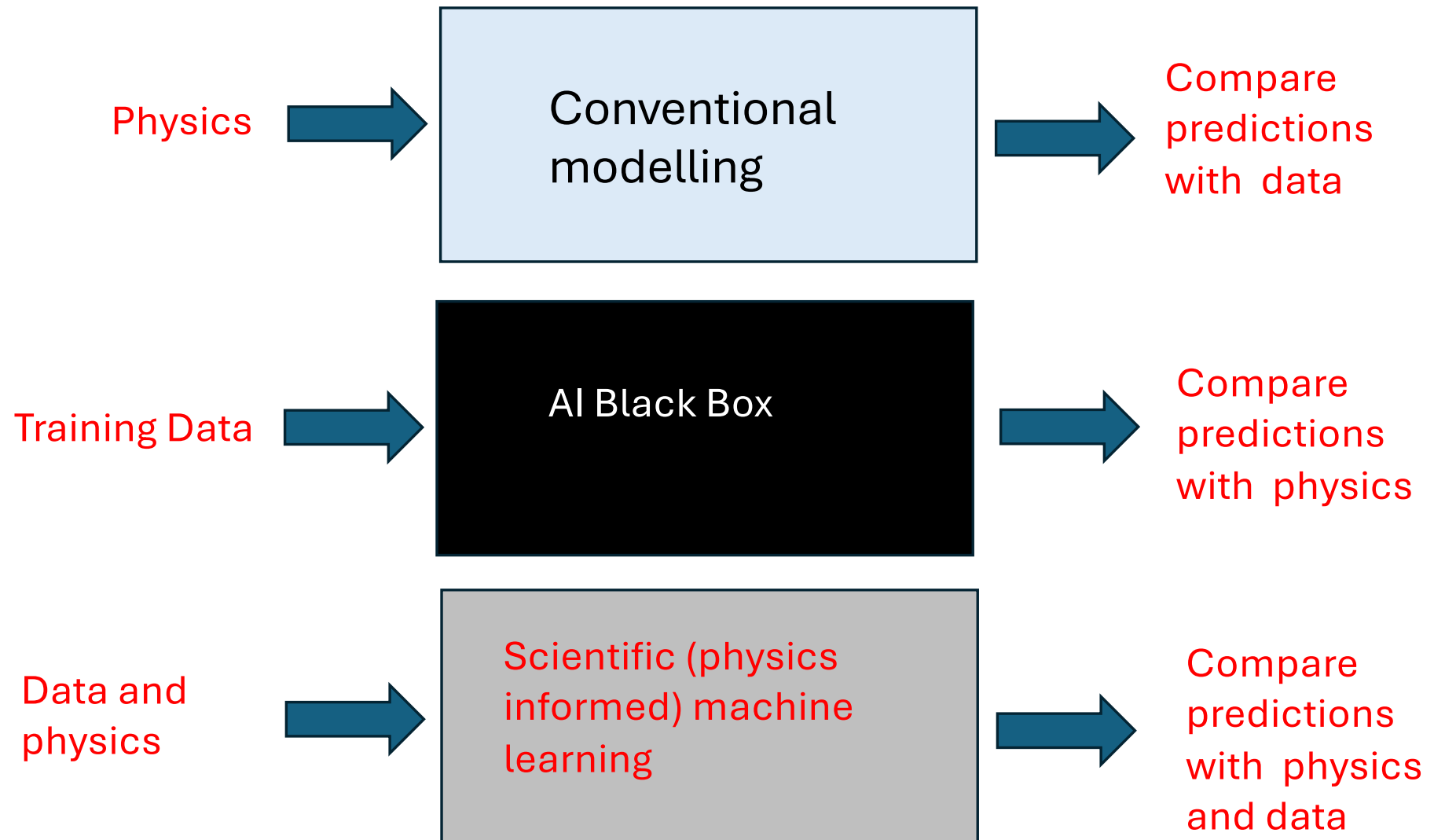
6 Case Study: PINNS



We are seeing a major impact of machine learning mathematics, physics, engineering, and business

- Classification and identification
- Solution of differential equations (ODEs and PDEs)
- Simulation of differential operators
- Solution and regularization of inverse problems
- Prediction and time series analysis
- Classification and generation of images (down scaling)
- CAD
- Forecasting the weather (from data alone!)





Scientific Machine Learning

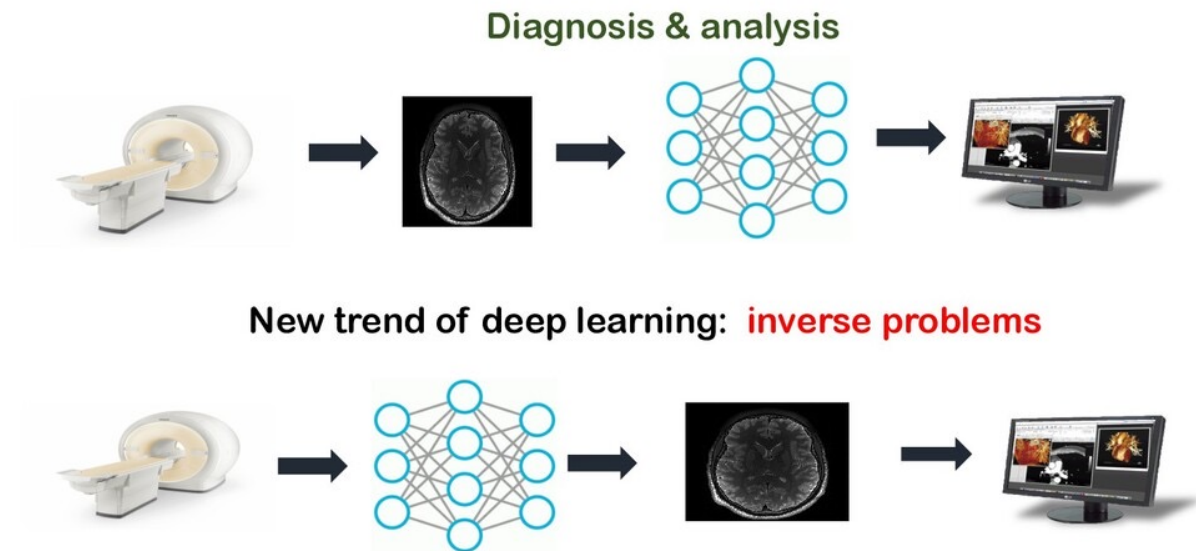
Aims:

- Derive
 - Effective
 - Efficient
 - Transparent
 - Reliable
 - Unbiased (data, inductive, learning)

Machine learning methods informed by physics and other disciplines

- Prove (where possible) rigorous results on their accuracy, stability, and convergence
- Apply them to reliably solve **important scientific and engineering problems**

Deep Learning for Inverse Problems



- A brief introduction
to the world of
(scientific) machine
learning



Image classification/Prediction

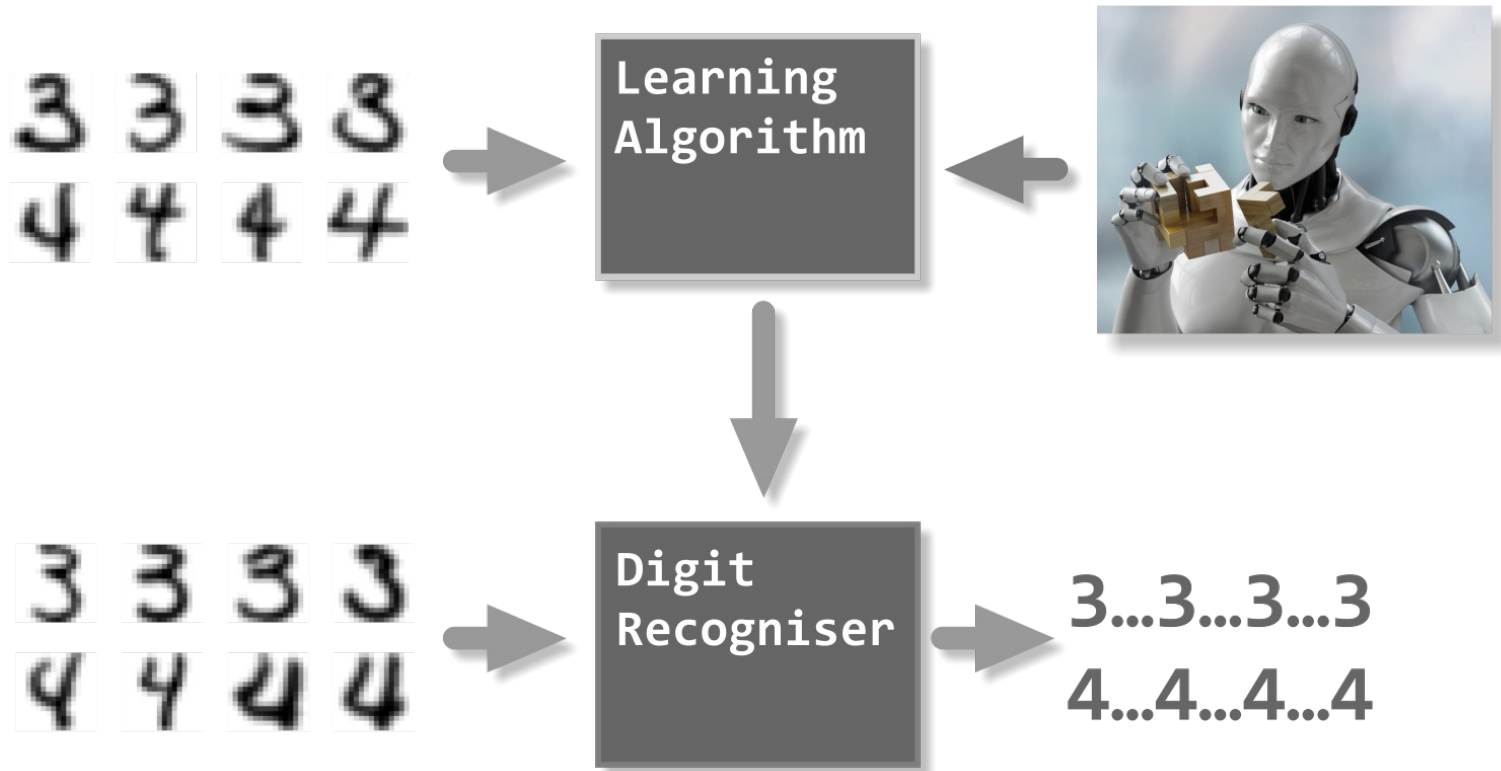
Probability distribution of images

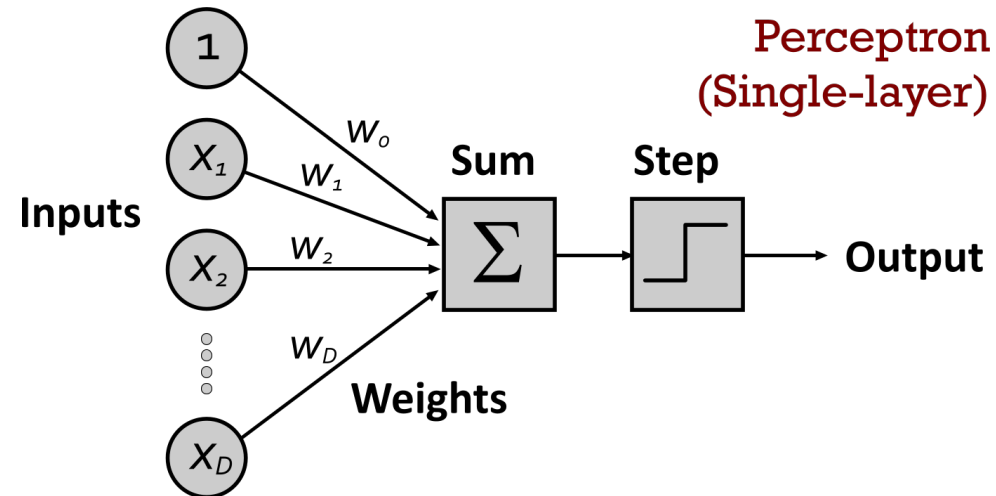


Neural net trained on
samples from the set of
images



Image label

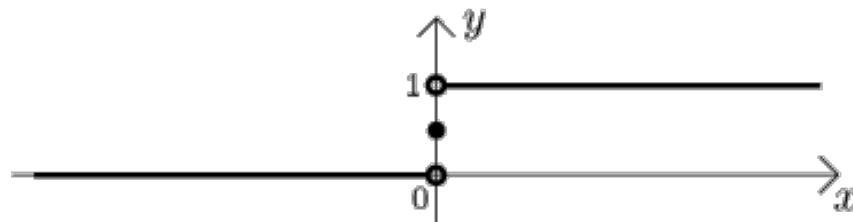




Breakthrough in classification came with the invention of the
neural net (1963)

$$Out = H \left(\sum_{i=1}^D w_i X_i - C \right)$$

$$H(x) = \begin{cases} 1, & x > 0; \\ \frac{1}{2}, & x = 0; \\ 0, & x < 0. \end{cases}$$





Example: Can you tell a cat from a dog?

Make

two

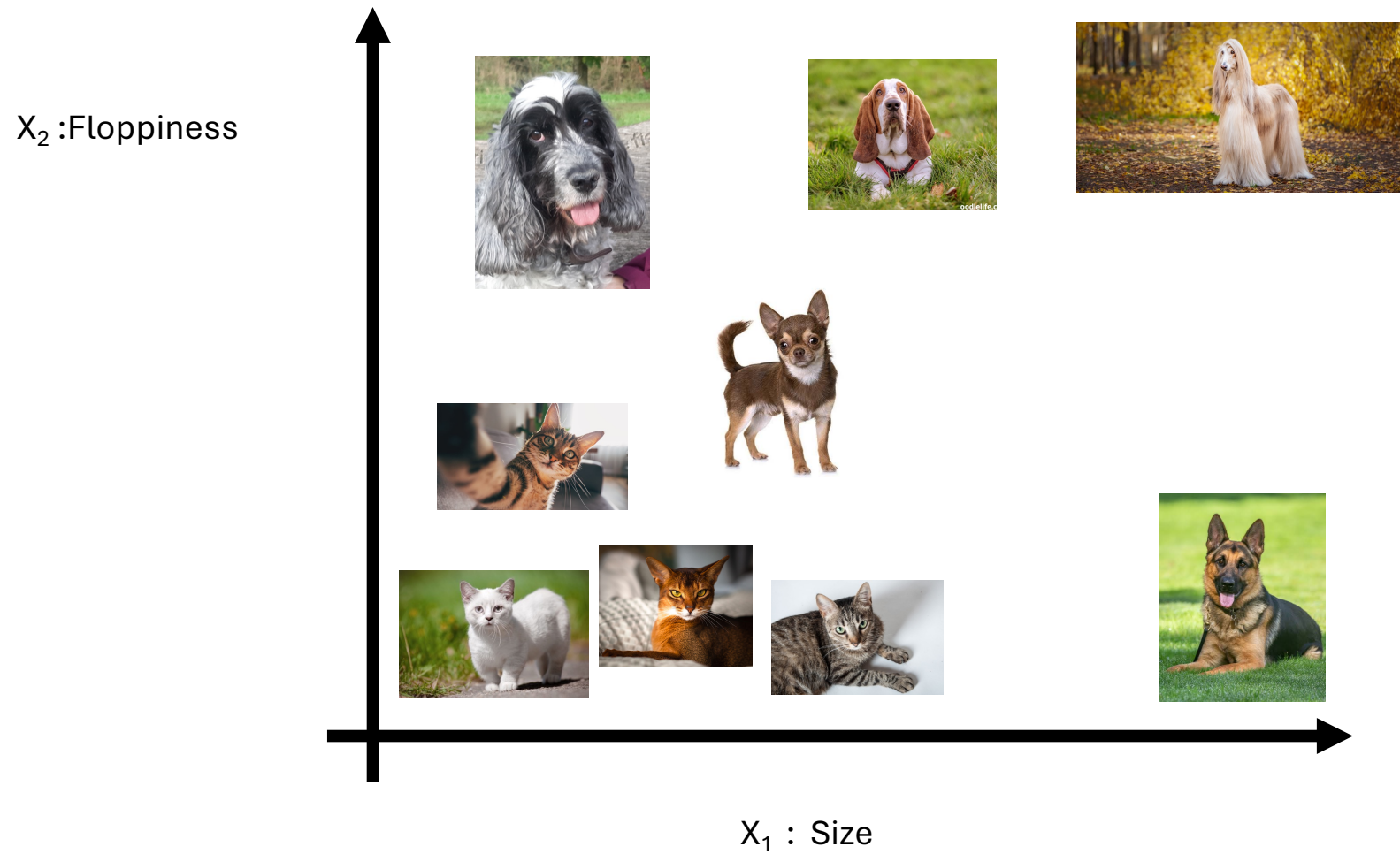
measurements

X_1 : Size

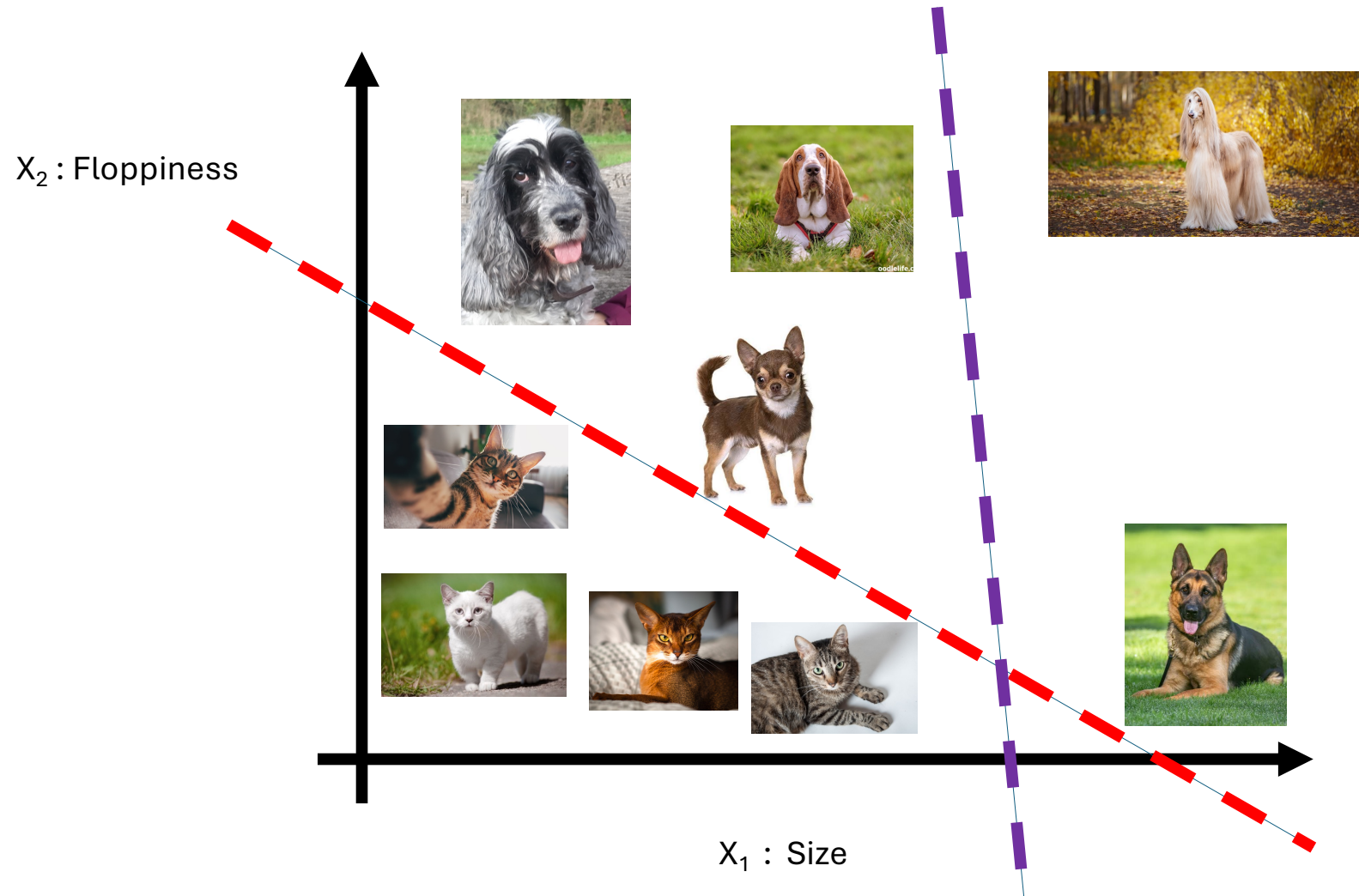
X_2 : Floppiness



Measure size and floppiness for lots of cats and dogs and plot



Draw a line which separates cats from dogs



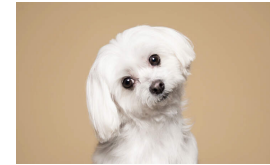
What is going on?

Equation for the line is:

$$w_1X_1 + w_2X_2 - C = 0$$

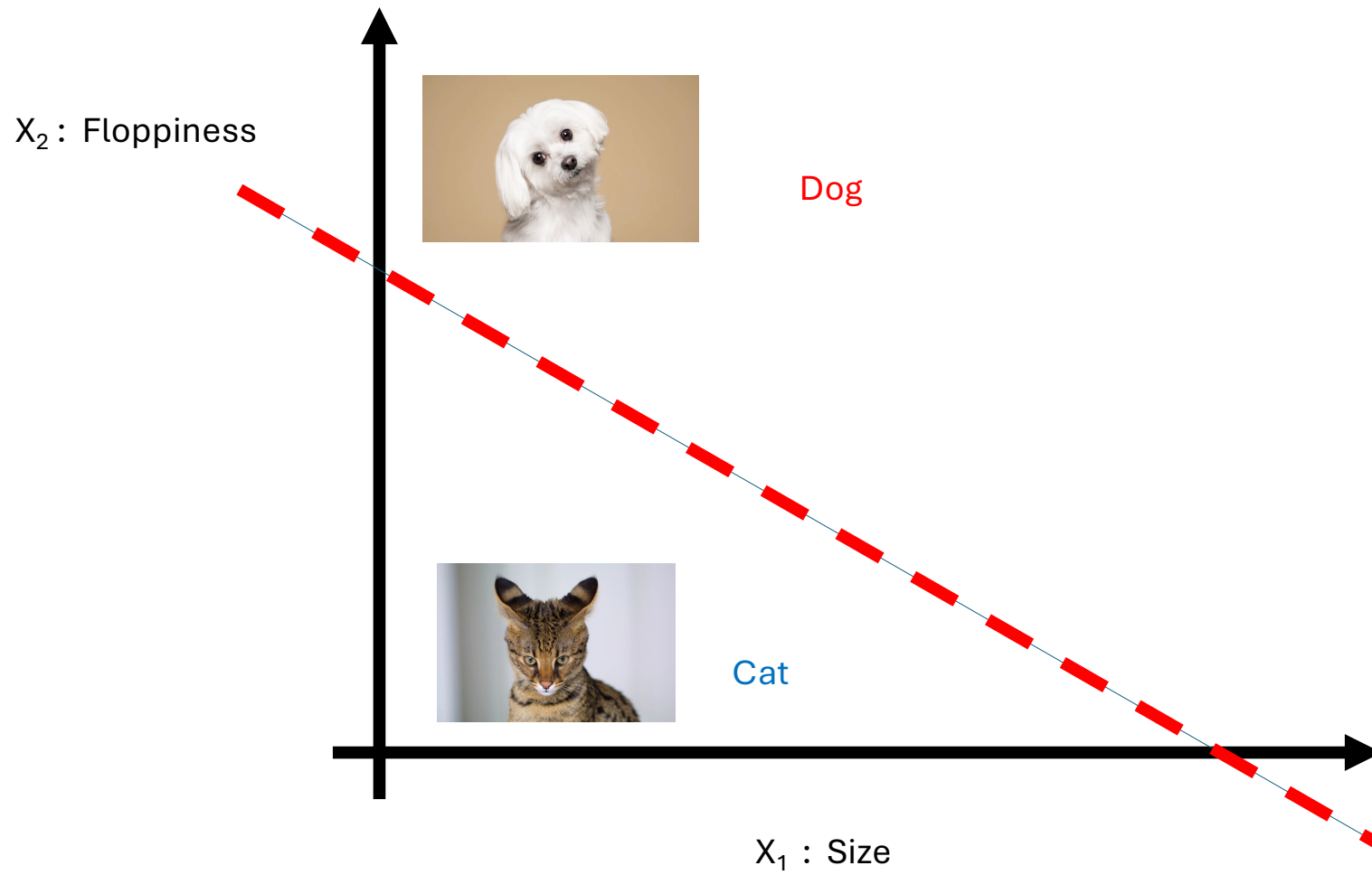
$$w_1X_1 + w_2X_2 - C > 0 : \text{Dog}$$

$$w_1X_1 + w_2X_2 - C < 0 : \text{Cat}$$

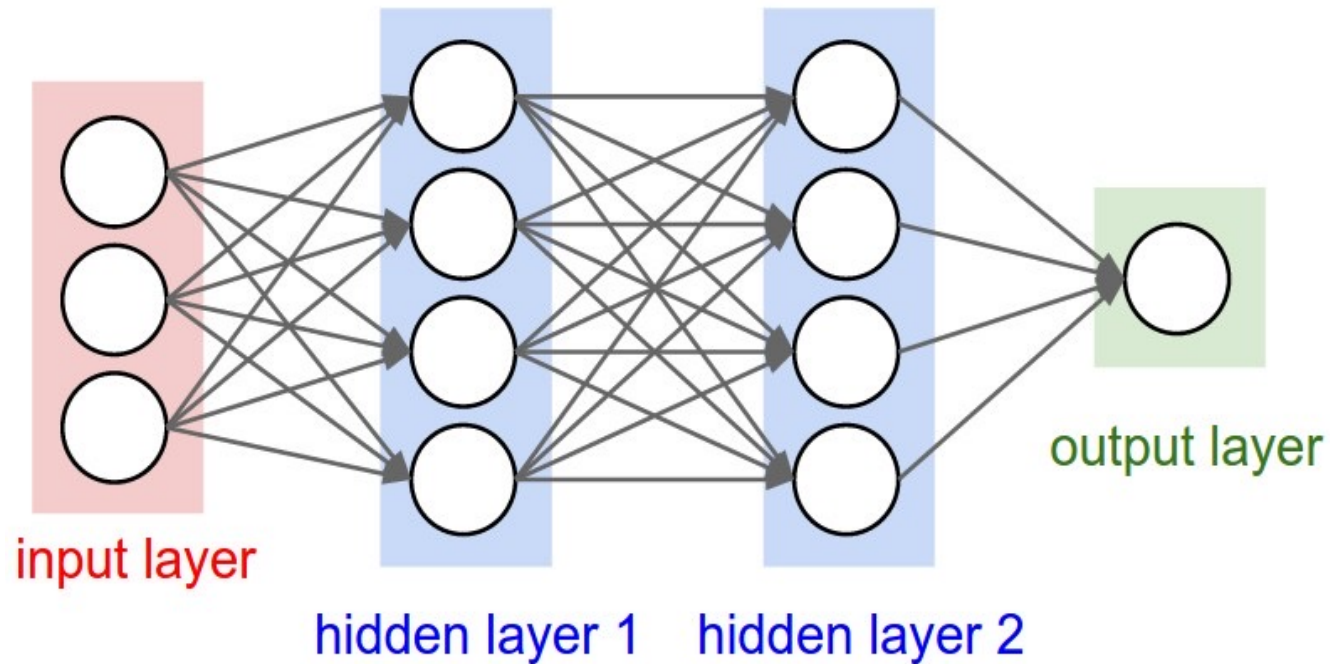


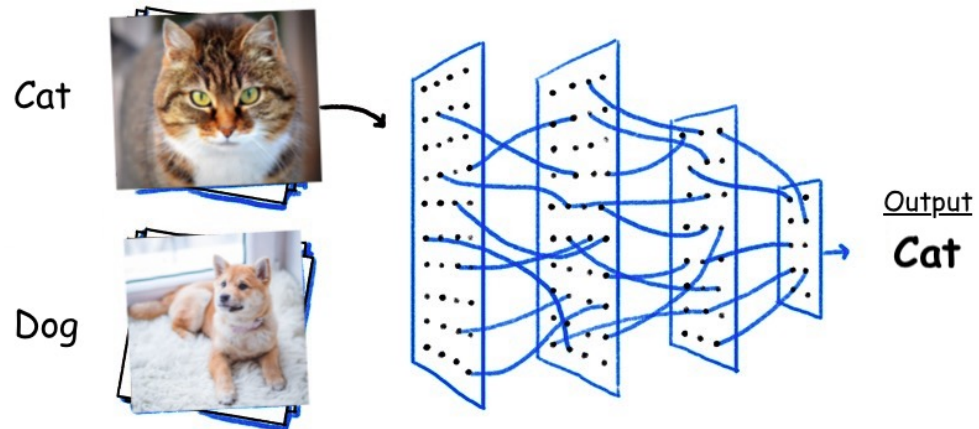
Train the **weights** w_1, w_2, C on the data to give the right result

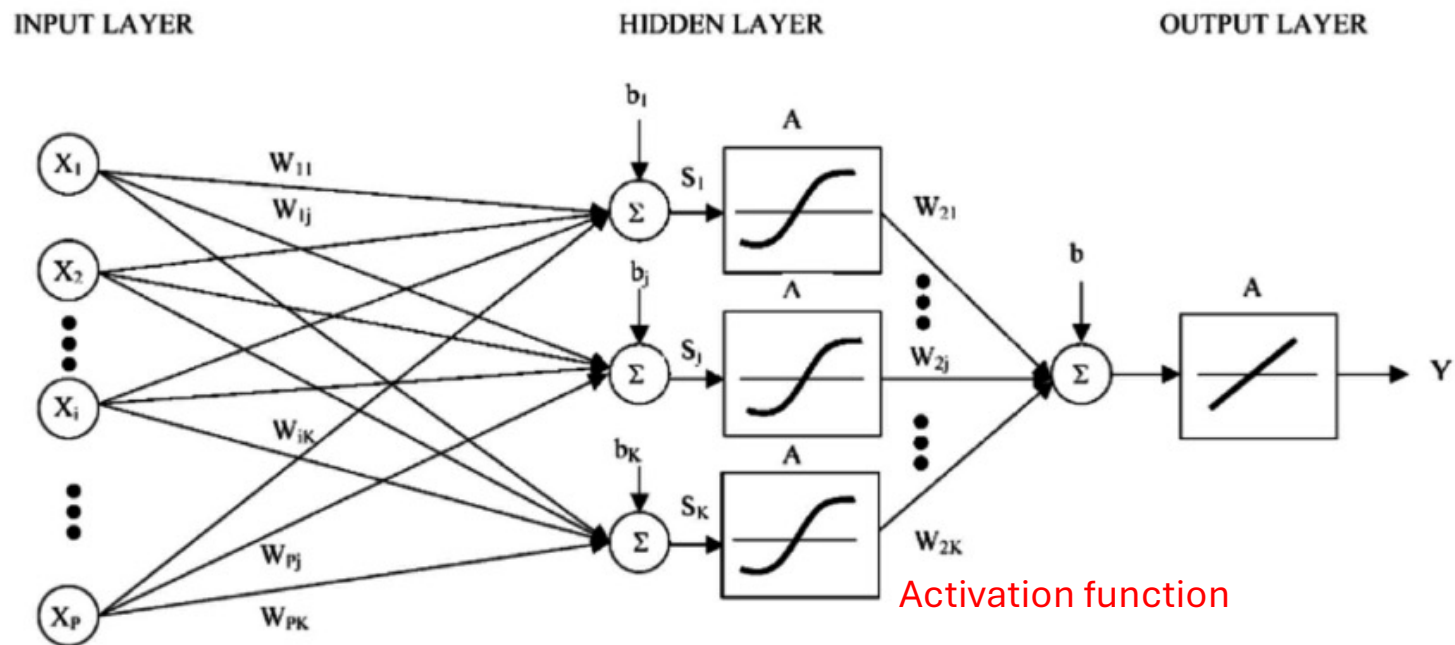
Can now tell a cat from a dog



Deep neural nets (2010) use many more layers and can be used to make much more complex decisions







Feed forward neural network

Or more mathematically, using a notation we use later on:

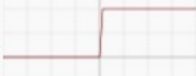
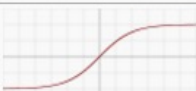
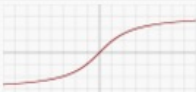
$$y_{j+1} = \sum_{i=0}^{W-1} c_i \sigma_{i,j} (a_{i,j} y_j + b_{i,j})$$

Input: $y_0 = x$. Output: $y(x) = y_L$

W: Width L: Depth

$\sigma_{i,j}$: Activation functions. (usually nonlinear)

θ : Set of All of the parameters

Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parametric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) [3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$

Need to find the best values for the Parameters

Achieve this by training the NN using a training set of data and a measure of success

Can take many hours of computer time on a GPU.

With a vast expense of energy!!



Mathematical Ingredients of the training process

Loss function $L(W)$: How close is the answer

Back propagation: How does the answer depend
on changes in the parameters.

Achieved by using automatic differentiation

PyTorch does this with ease!

Optimisation: Choose the parameters to

minimize the loss over (a subset of the)
training data

Validation: Test on separate data to avoid
over fitting to the answer

Optimise the weights using optimisation methods:

Eg. stochastic gradient descent, ADAM, quasi-Newton, simulated annealing, Monte-Carlo Tree Search (MCTS)

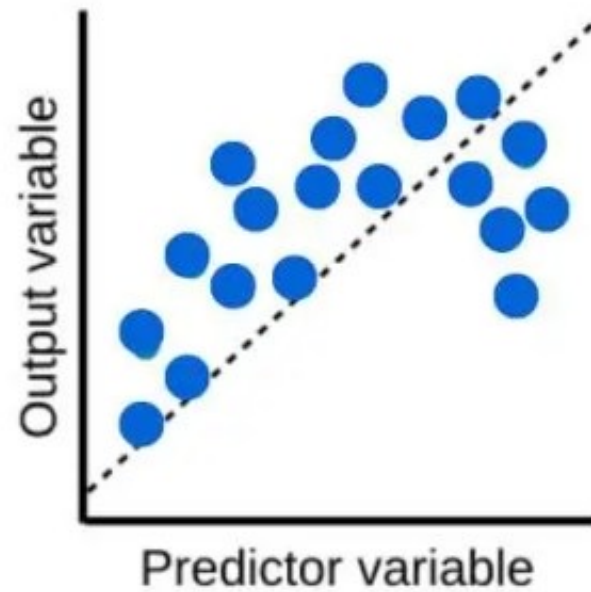
Implemented in eg. PyTorch
(see the Case studies)



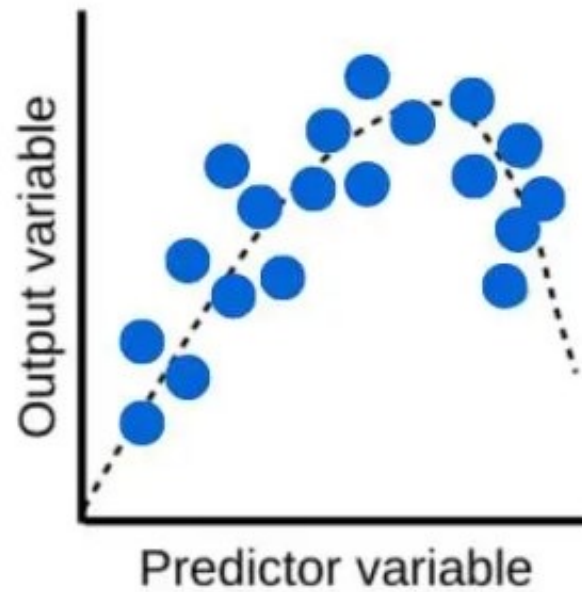
Tensor Flow/Keras

AlphaZero used reinforcement learning over 700,000 games with MCTS optimisation

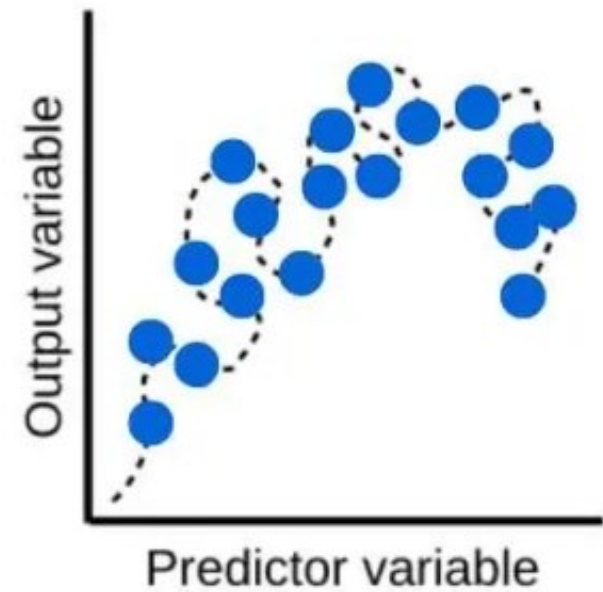
Underfitting

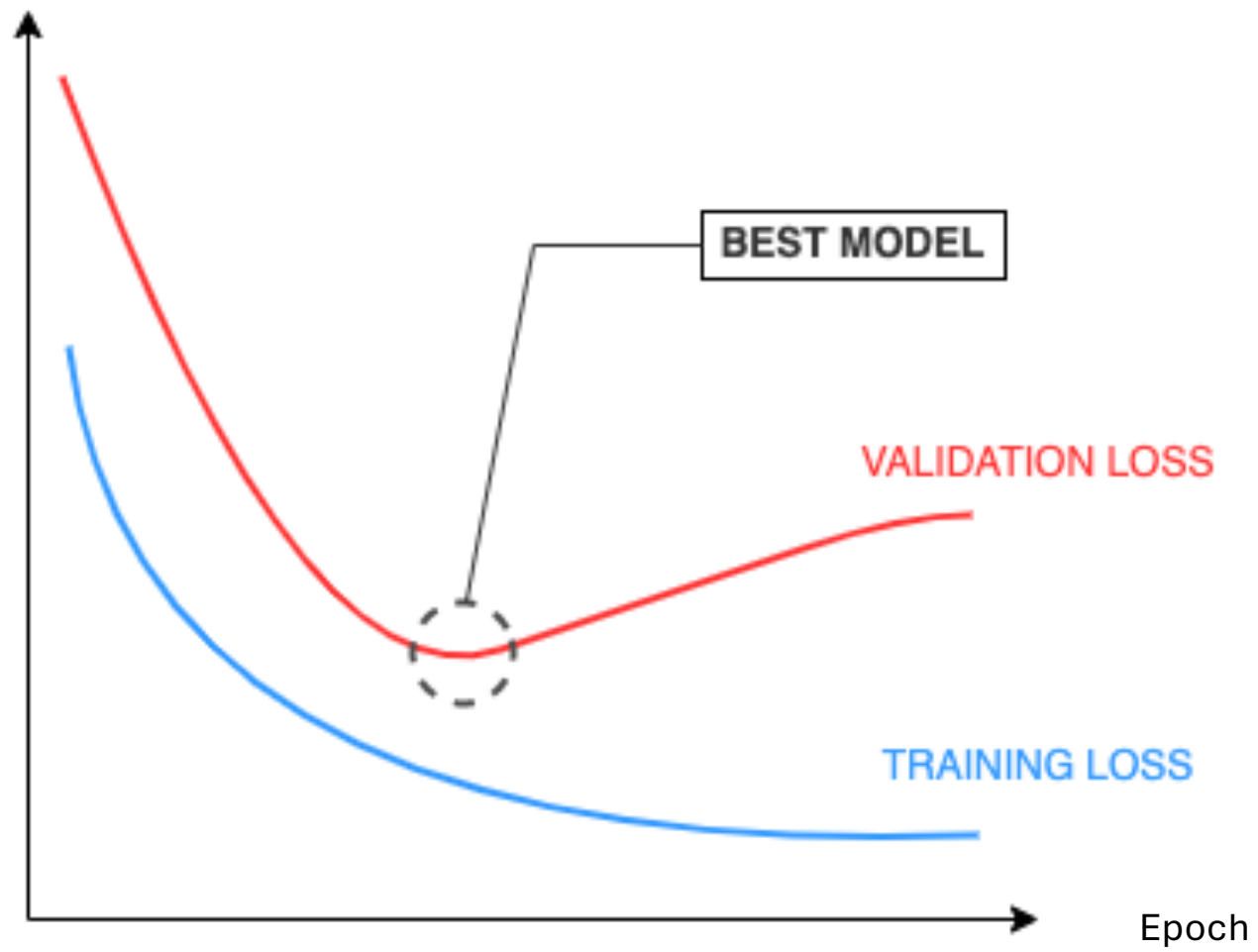


Optimised Fit



Overfitting





- Supervised training

Have a **labeled training set**

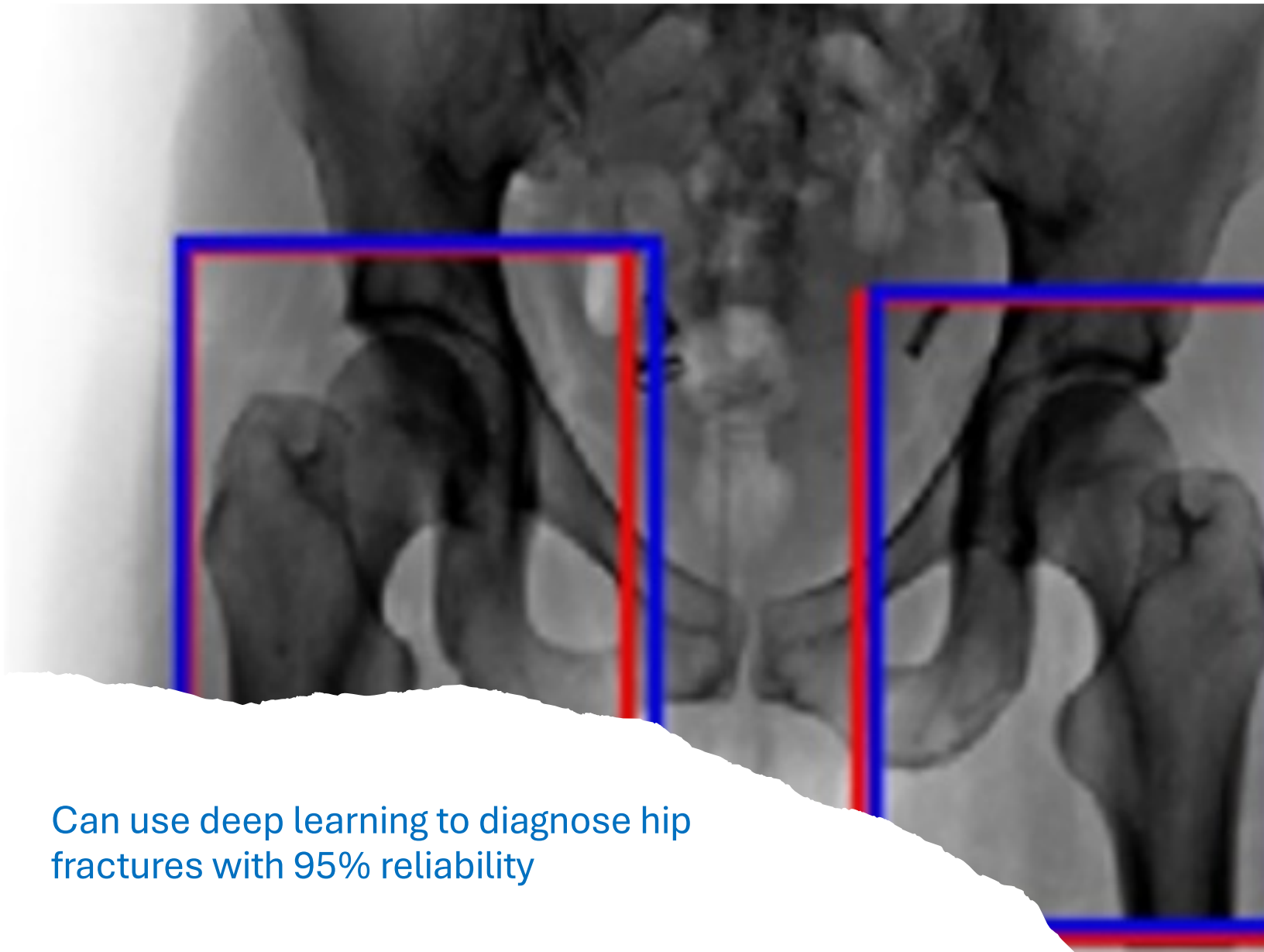
eg. cats + dogs, musical genres,
Inverse problems, approximation

- Reinforcement training

Have a measure of success

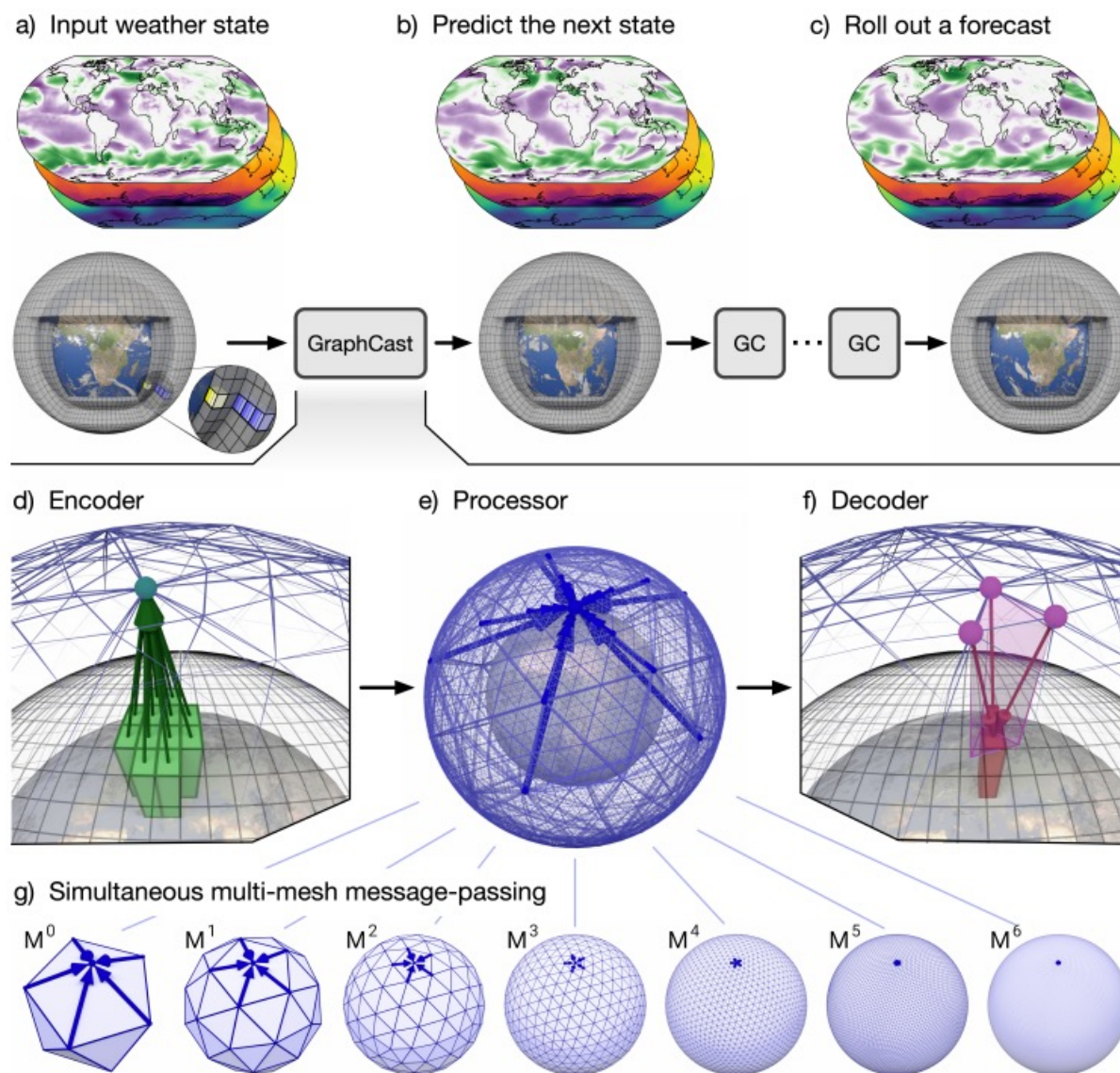
eg. winning GO, solving an equation
(eg. a PINN), variational method





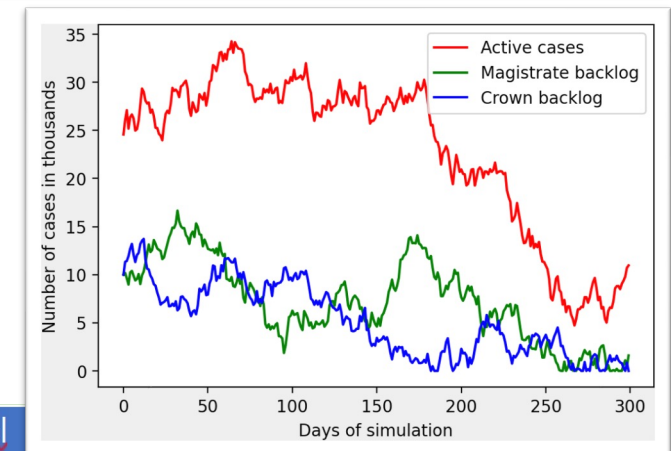
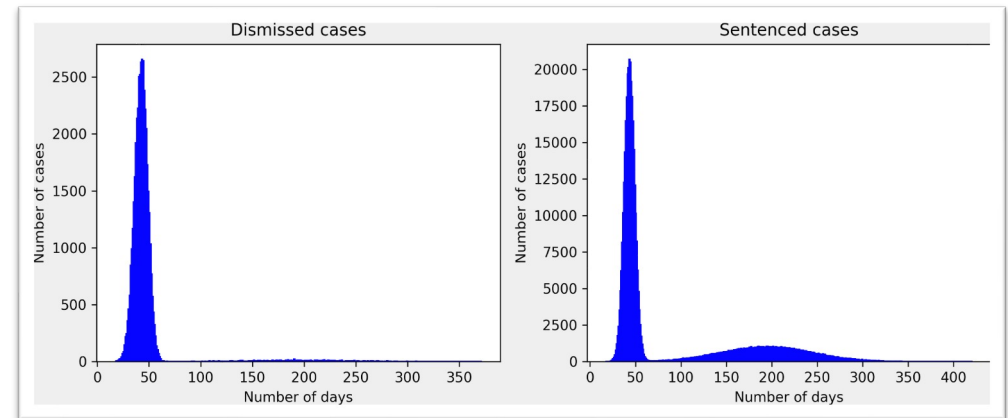
Can use deep learning to diagnose hip fractures with 95% reliability

Weather forecasting trained on the ERA5 Data Set



Maths and AI for Justice

NN can predict case volume and duration in the judicial system based on case characteristics



Time distribution: Lognormal

Time spent in summoned: 34 days

Summoned to Exit: 9 days

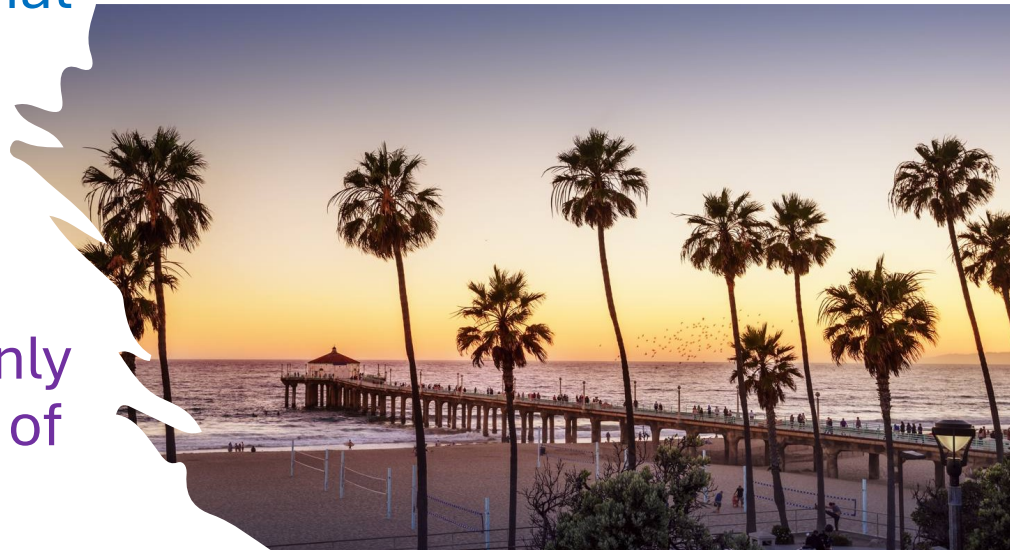
Time spent in Sentencing: 7 days

Data: the national statistics for the Criminal court statistics quarterly (April - June 2023)

But ..Can a weather
forecast trained on
Californian data
predict a storm in the
UK?

Learning only from data
makes it hard to make
decisions away from that
data

Would you trust a
computer to make a
judicial judgement if only
trained on some types of
cases?





Also:

Some images are hard to
classify correctly



Machines can get it wrong

Identified as a 45mph speed sign



“panda”

57.7% confidence

+ .007 ×



noise

=



“gibbon”

99.3% confidence

And can get confused by an
‘adversarial attack’

Study this as a conditioning
problem in numerical analysis

Machine learning models can memorise data: our privacy is at risk

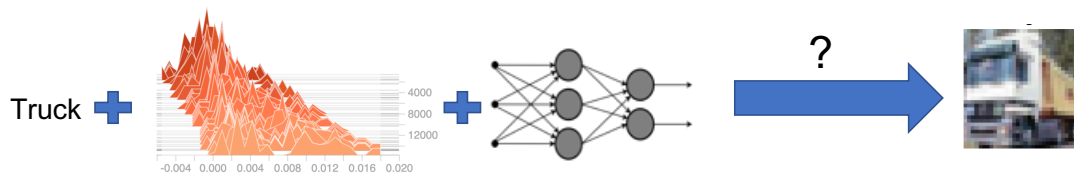
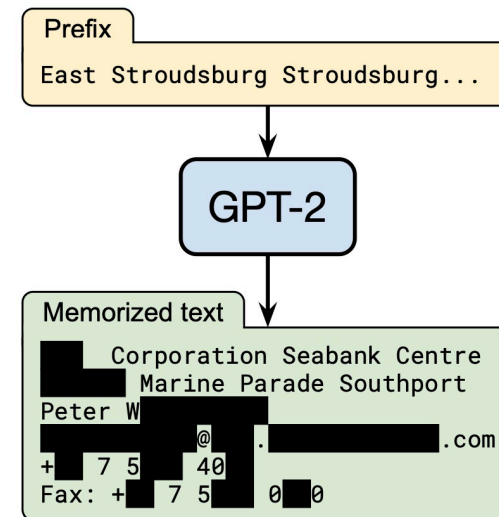


Image classification models can **memorise their training images** through the training updates (i.e. gradients).



Language models can also **memorise their training** can be extracted using text prompts.

Chen C, Campbell N. Analysing Training-Data Leakage from Gradients through Linear Systems and Gradient Matching, The 33rd British Machine Vision Conference (BMVC), 2022

Carlini, Nicholas, et al. "Extracting training data from large language models." 30th USENIX Security Symposium (USENIX Security 21). 2021.

One way to mitigate the privacy risk: training with differential privacy

